SoC Labs · University of Southampton

MegaSoC

FPGA Prototyping & Hardware Bring-Up

Consolidated Work Record — Revision 6

Field	Detail
Prepared by	Anastasios Dimitriou — ad1u25@soton.ac.uk (with Claude Code, Anthropic, as bring-up assistant/co-author for Revisions 5 and 6)
Role	Senior Research Hardware Engineer, SoC Labs
Supervisor	Daniel Newbrook — d.newbrook@soton.ac.uk
Document date	21 October 2026
Period covered by this revision	29 July 2026 – 21 October 2026. Sections 3–13 are Revision 5 (to 30 September) carried over in full with update notes; Sections 14–20 cover 1–21 October 2026.
Design	MegaSoC — Arm Cortex-A53 SoC, SoC Labs
Platform	Synopsys HAPS-SX VU19P 1F (xcvu19p-fsva3824-2-e); Vivado 2024.1 on the Linux server (sole build machine since 12 October); bench PC haps-dev for the two UARTs
Repository	git.soton.ac.uk/soclabs/megasoc_project — FPGA_Tasos/haps_top/
Tape-out milestone	1 October 2026, IMEC
Headline change since rev3	The single open blocker in rev3 — "CPU never asserts ARVALID, root cause unknown" — is RESOLVED. Root cause found and fixed. Reliability confirmed at 100% (164/164 boot attempts, two independent hardware captures) once an unrelated measurement bug was also found and fixed.
Headline new work opened by this revision	A design-wide hold/removal timing audit (prompted by an independent constraints review) found the ARVALID fix was masking ~157,000 further real, previously invisible timing violations. Two of three root causes were fixed by 17 September (Section 5). The third, the QSPI clock, was resolved between 17 and 30 September by reclocking the controller on the system clock (Section 9.6); setup is now met with margin and the design has one clock domain.
Headline of the 17–30 September period	A structured review of every project-authored source, the synthesis and implementation logs and the routed netlist found 57 issues (Section 9 and the bug register in Section 10). The largest remaining problem is hold: about 20,000 endpoints failing by small amounts from clock skew across a design placed over two SLRs at 6% utilisation (Section 9.10). The boot console has still never been observed on hardware (Section 4.3), and no bitstream produced after early September has been run on the board.

Headline of the 1–21 October period (this revision)	The FPGA bring-up is complete. The boot console was observed for the first time on 10 October (build 42) once a polarity bug in the FPGA SRAM wrapper was found with the whole-bus logger (Section 14). Seven of the eleven SoC Labs tests then passed from RAM loaded over UART (build 48, 15 October), from an in-FPGA flash emulator (build 49) and from the physical on-board flash (build 51, 20 October) after two bugs in the QSPI controller's 1-bit write path were found and fixed (Section 17). The last blocker, a stop at cache enable present since build 42, was root-caused by RTL simulation of the FPGA design to Arm's TSMC example L2 tag RAM integration (written for a 1 MB L2, against this core's 256 KB) and fixed in build 54; on 21 October hello_world booted from the physical flash with both CPU caches on, the stock boot sequence end to end (Section 18).
Findings relevant to the ASIC from this period	Two bugs in the shared QSPI controller RTL (N39, one extra clock on every 1-bit write; N27, bit order within nibbles): no flash that needs a 1-bit status write can be programmed from the SoC until they are taken upstream. Firmware: scatter-loading never runs (N35, with an uninitialised global in the flash boot path); SPI_WAIT_BUSY can hang after a short command (N38); no Winbond branch in SPI_STARTUP (N26, validated branch supplied). Test code: uart_tests cannot pass on hardware (N40). Sections 17 and 19, bug register rows 51–68.

1. About this revision

This document is Revision 6 of the MegaSoC FPGA bring-up work record. It contains Revision 5 (30 September 2026) in full as Sections 1–13 — itself containing Revision 4 as Sections 1–8 — with update notes in italics where later work changed the position, and adds Sections 14–20 for the period 1–21 October 2026. Section 19 supersedes Section 11 as the current status. It supersedes nothing in Revisions 1–3 (consolidated as "MegaSoC_Work_Record_rev3.docx", dated 29 July 2026) — that document remains the authoritative history of everything done before 29 July 2026, and Section 2 of this revision summarises it rather than repeating it in full. This revision picks up exactly where rev3 left off and covers everything done since. Section 10 is a consolidated bug register for the whole programme: one row per bug, giving the bug, its root cause and the fix where one is implemented, with no reference to build numbers or source-tree versions; Section 10.7 adds the rows found in October.

Conflict resolution rule, carried over from rev3: where sources disagree, the most recent hardware evidence wins, and the superseded position is recorded explicitly (Section 7) rather than silently dropped. This revision overturns one of rev3's own headline numbers — the measured ARVALID reliability rate — and the reason it changed is itself one of the more important findings of this period (Section 3.3).

1.1 Source documents consulted

Source	Date	Role in this revision
MegaSoC_Work_Record_rev3.docx	29 Jul 2026	Authoritative prior history. Summarised in Section 2; its one open item is closed in Section 3.
MegaSoC_FPGA_Handover_Report.docx	Jul 2026 (rev 2)	Superseded by rev3; consulted only to confirm rev3's consolidation was complete and accurate.
MegaSoC_FPGA_Complete_Summary(_2).docx	Jun–Jul 2026	Superseded by rev3 (Q-channel root cause, later disproven). Consulted for extra technical detail (exact ILA capture values) not carried into rev3's condensed table.
MegaSoC_FPGA_BringUp_Complete.docx	Jun 2026	Earliest full log. Superseded on clock frequency, pin assignments and ILA status. Consulted for the full list of 12 SRAM macro names.
MegaSoC_FPGA_Work_Record.docx	27 Jul 2026	Programme/Jira context (task breakdown, owners, licensing, X2Go environment issues) not restated in as much detail in rev3. Summarised in Section 2.1.
PLL_Integration_Summary.docx	Jun 2026	DDR PHY PLL investigation, explicitly out of the author's main scope in every prior report. Summarised briefly in Section 2.6 for completeness only.
Interactive bring-up work, Linux server	7–17 Sep 2026	Sections 3–8 (carried over from rev4).
MegaSoC_HAPS_Bug_Register_2026-09-29.docx; external review note of 23 Sep	23–29 Sep 2026	Structured review of the project on the Windows workstation; 57 items, folded into Sections 9 and 10. The 23 Sep note drove the Linux-side changes and is superseded by the register.
Interactive bring-up work, both machines	17–30 Sep 2026	Sections 9–13.
MBT Jira board (soclabs.atlassian.net, project MBT)	29 Sep 2026	Tickets MBT-23 to MBT-36; cross-referenced in Section 10.
Interactive bring-up work, Linux server and bench PC haps-dev	1–21 Oct 2026	Sections 14–20. Per-build notes are archived with each bitstream under FPGA_Tasos/haps_top/bitstreams/build43_… to build54_….
MegaSoC_MBT_Jira_Updates_2026-09-30.md (running log, Desktop)	9–21 Oct 2026	Status proposals for MBT-23…36 and new tickets N24–N40 with evidence; Atlassian was unreachable from the server, so the text is paste-ready. Folded into Sections 10.7 and 19.
RTL simulations of the FPGA design (VCS) and unit testbenches (xsim)	14–21 Oct 2026	Sections 15.4 (QSPI controller), 16.1 (RAM slave), 17.1 (flash emulator), 18.2–18.3 (cache stop).

2. Summary of prior work (Revisions 1–3, December 2025 – 29 July 2026)

Full detail is in MegaSoC_Work_Record_rev3.docx. This section is a condensed recap, included so this document stands on its own.

2.1 Programme and environment

MegaSoC is a three-year SoC Labs programme to design and fabricate a large, Linux-capable Arm-based SoC. The first milestone is a reduced-functionality tape-out to IMEC, deadline 1 October 2026 (Ethernet, PCIe and the chiplet interface are out of scope for this tape-out). The FPGA prototype's job is to prove the reduced SoC boots and behaves correctly before that database is committed. All work is tracked under Jira project MGSOC, parent task MGSOC-18. A dual-connection remote workflow (VS Code Remote-SSH, MobaXterm for X11, later an X2Go/XFCE desktop with its own diagnosed faults) was established against the development server srv03335.soton.ac.uk. Vivado ML Enterprise Edition licensing (the XCVU19P requires it; the free Standard edition does not cover this device) and a four-part VCS/Verdi simulation environment were both set up and documented as prerequisites.

2.2 Making the ASIC RTL synthesisable

- Twelve foundry SRAM macro behavioural models written (11 RFHSD_PG_SVT_* 1-cycle, 1 SRAMHDE_PG_SVT_* 2-cycle) — the A53 instantiates these by name and synthesis fails outright without them.
- Two cache/TLB RAM width parameter fixes: CPU_CACHE_PROTECTION=1 (80→84 bits) and CA53_SCU_L2_TAGRAM_DATA_W=40.
- Clock-gate cell PREICG_X4B_A9TS_C31.sv modelled from scratch (this file is revisited heavily in Section 4 of this revision).
- Clock controller bypass: FPGA_BUILD ties all system clocks directly to the single board clock for a synchronous bring-up domain; without this Vivado pruned most of the SoC as constant-driven logic.
- A 44-page platform reference document was produced for the HAPS-SX VU19P 1F board, because no consolidated textual pin map existed anywhere in the vendor documentation set.

2.3 Board bring-up and the elimination sequence

Pin assignments were corrected in three places (UART moved from unrouted/LED-conflicting pins to PMOD3); ILA debug infrastructure was built up to 16 probes; a ROM Q-channel deadlock was diagnosed and fixed (genuine bug, retained) but later shown, once observed against the fixed RTL, not to be the cause of the boot failure — the captured stall had been observed on a bitstream that predated the fix. The design's actual clock frequency was corrected from an assumed 20 MHz to the real ConfPro setting of 100 MHz, then deliberately reverted back toward a lower frequency later for reasons specific to this SoC's own clock-gating structure and JTAG margin (see Section 4.3 of this revision, which documents the current, different, considered choice made in the XDC actually in use for the builds in this revision). By the end of rev3, every mechanism between power-on and the CPU's first bus transaction had been individually verified working with direct hardware evidence — clock, reset, ROM Q-channel, CA53 execution clock toggling, interconnect decode, boot ROM content — except one: the CPU's own AXI master port never asserted ARVALID. That single open question, MGSOC-33, is what Section 3 resolves.

2.4 Bugs found and fixed before this revision

#	Bug	File	Consequence if unfixed
1	ROM Q-channel ports tied off	megasoc_tech_wrapper.sv	PPU deadlock, ROM domain never sequenced on (real bug, later shown not to be the ARVALID cause)
2	SRAM external-gate override wired to the live power request	SRAM_wrapper.sv	Low-power sequencer could park, blocking all SRAM AXI traffic
3	Console output routed to a debug-only peripheral (USRT0 not UART0)	uart_stdout.c	No UART output would ever appear on the board pins
4	XDC clock period wrong relative to ConfPro's actual setting	haps_top.xdc	Design analysed at the wrong frequency
5	Clock-gate cells synthesising as LUTs on the clock tree	PREICG_X4B_A9TS_C31.sv	141 DRC critical warnings; ~20,000 failing setup paths at the time
6	Boot ROM image stale relative to a firmware fix	bootrom.v	A UART fix would never have been compiled into what the FPGA ran
7	Bootrom writes UART before configuring the GPIO pin mux	firmware	Latent silicon bug — early output lost on the real chip

2.5 Status at the end of rev3 (29 July 2026)

Verbatim from rev3's own headline:

"Design builds, programs and runs. CPU is powered, clocked and out of reset but never issues its first instruction fetch. Root cause open."

Every other subsystem checked was ruled out with direct hardware evidence: clock path, reset path, ROM Q-channel handshake, CA53 execution clock (confirmed toggling via a properly synchronised 2-flop toggle detector), boot ROM content (hand-decoded as valid AArch64), interconnect decode, and a full simulation boot through the SRAM memory test. The one untested lead going into this revision's period was the CPU_L2/CPU_NEON power/reset domains, sequenced by their own independent PCK-600 PPU instances.

2.6 DDR PHY PLL (excluded scope, noted for completeness only)

A separate task (dev_lpddr4_integration branch) to use the Synopsys DDR PHY's dwc_ddrphymaster_top as a behavioural clock source. The PLL's CSR registers are normally programmed by the PHY's own embedded PMU firmware (loaded from ICCM); tying CSR ports to constants does not configure it, so the PLL never locks that way. The delivered, working solution is a hybrid megasoc_clock_ctrl.v: self-oscillating clocks as the working source, with the real PLL instantiated and monitored in parallel and a USE_PLL_CLOCKS parameter to switch over once PllLockStatus asserts. This work is explicitly out of scope for the rest of this document, per the author's standing instruction in every prior report.

3. The ARVALID root cause — found, fixed, and confirmed at 100%

This section resolves MGSOC-33 and the single open item rev3 ended on. It is the headline result of this revision.

3.1 Root cause

The CPU's execution clock enable flop (ca53governor_cpu_clk_reset.v: stop_clk_reg) has an async PRE (preset) pin fed by a resync flop (u_ca53_cell_sync0/r1_reg) elsewhere in the same governor module. The timing relationship between these two flops includes a Removal check — the async-control-pin analogue of a hold check — which needs the data path delay between them to be large enough, not small (the opposite intuition to an ordinary hold/setup problem).

The XDC contained a blanket waiver, present from early bring-up and described in every prior report as “standard for large SoC prototypes on single clock”:

```
set_false_path -hold -from [get_clocks HAPS_CLK]
set_false_path -hold -to [get_clocks HAPS_CLK]
```

In Vivado, a -hold-class exception covers both ordinary Hold and Removal analysis. Because HAPS_CLK is the only declared clock in the design, this pair of lines silently excluded Removal analysis from this specific path — and, as later became clear (Section 5), from essentially every hold-type check in the whole design. No build had ever actually verified this path was safe; different builds' placement just happened to land on different sides of the real, unconstrained margin, which is why an earlier build (build 6, prior revision) had appeared to work and a later, nominally identical build (build 13) did not.

3.2 The fix: a placement pblock

Because the exception-precedence problem could not be solved by rewriting the timing exception itself (set_max_delay/set_min_delay were confirmed, empirically, to never override an existing set_false_path on the same path in this Vivado version, regardless of specificity), the fix actually used is a placement constraint instead — it cannot affect any other path's hold-waiver coverage, since it only tells the placer where two specific cells may go:

```
create_pblock pblock_governor_clkgate_proximity
resize_pblock pblock_governor_clkgate_proximity -add
{SLICE_X170Y440:SLICE_X230Y500}
add_cells_to_pblock pblock_governor_clkgate_proximity [get_cells -hier -filter { \
  NAME =~ "*u_governor_cpu_clk_reset/u_ca53_cell_sync0/r1_reg" \
  || NAME =~ "*u_governor_cpu_clk_reset/stop_clk_reg" \
}]
```

Several pblock sizes and an explicit LOC-pinned placement at the same measured distance were tried (20×20, 200×200, and a hand-picked site pair on the pblock's own best-measured row). None beat the original 60×60 region's own free-placement result; all were reverted. The lesson recorded from that experiment: Removal margin did not correlate monotonically with physical cell distance (+0.085 ns at 54 columns apart, −0.034 ns at 76 columns, −0.131 ns at 1 column) — clock-tree skew, not raw

distance, turned out to be the dominant term (this is explained structurally in Section 5.3). The 60×60 pblock, exactly as first written, is the configuration that is proven and retained.

3.3 The measurement bug that was hiding the true reliability rate

With the pblock in place, repeated hardware boot-cycle captures over several builds appeared to show roughly 78% reliability (28/36 pooled attempts asserting ARVALID/RVALID), which is what earlier iterations of this revision's own investigation initially reported and acted on — including a written summary sent externally describing a residual ~22% failure rate and several follow-up hypotheses (a second, structurally similar hazard in ca53_cpu_reset.v; the SCU tag-invalidate handshake). Both of those leads were investigated with real hardware correlation testing and found weak or inconclusive.

The actual explanation was found by accident while investigating those leads: the debug-UART capture process being used to gather this data was corrupted. Multiple orphaned cat /dev/ttyUSB0 processes, left running from earlier sessions across several days, were found still active and reading the same serial device concurrently, splitting the byte stream unpredictably between competing readers. This produced exactly the kind of garbled, dropped-byte log lines that had been quietly present throughout the whole investigation and were assumed, until this point, to be ordinary UART overrun.

Once every orphaned reader was killed and a single, clean capture was taken, the same 60×60-pblock configuration measured 51/51 boot attempts successful, then 113/113 on an independent repeat —

164/164 (100%) across two independent hardware captures.

The residual ~22% failure rate this investigation had spent real effort chasing (a second ca53_cpu_reset.v hazard, the SCU tag-invalidate handshake, various placement-tuning attempts) was never a real hardware defect. It was a measurement artefact of a broken capture process. This was independently reconfirmed on every subsequent build in this revision (builds 23 through 28 inclusive) that carried the same pblock: all measured 100%.

3.4 What this settles, and what it does not

- MGSOC-33 (“A53 boots correctly”), the gating hardware item in every prior report's checklist, is now supported by direct, repeated, clean hardware evidence: ARVALID and RVALID assert reliably, every boot, with the pblock fix in place.
- This says nothing about whether the CPU then executes correctly once it starts fetching — that is a separate question, addressed in Section 8.
- It also says nothing about the rest of the design's timing closure, which turned out to have its own, much larger, entirely separate problem — the subject of Sections 4 through 6.

4. Boot ROM content — verified and refreshed

A separate, shorter investigation, triggered by the natural next question once ARVALID was confirmed reliable: does the CPU then execute anything meaningful?

4.1 The FPGA “ROM” is a synthesised case-statement, not blank memory

ROM_wrapper.v (the copy actually compiled into this project, which has locally diverged from the shared megasoc_tech source tree) instantiates a bootrom module — an auto-generated Verilog case statement (bootrom_gen.py), not a $readmemh image, confirming the earlier finding that the ROM's contents are compiled directly into logic rather than loaded at runtime. The currently-embedded bootrom (dated 30 July 2026) was checked instruction-by-instruction: its first words decode as a standard AArch64 register-clearing startup sequence (0xd2800000 = MOVZ X0,#0, followed by a long MOV Xn,X0 run — a textbook C-runtime crt0 preamble), and the literal string “SoCLabs” is embedded in its data, confirming it is a real compiled image, not a placeholder.

4.2 Regenerated against the current bootloader build

bootrom_gen.py was re-run against the current bootloader.hex/SEC_ROM_BOOT_IMG. The result differed from the embedded bootrom by exactly one instruction word (plus the generation timestamp) out of roughly 8,200 — the embedded image was already almost exactly current. The freshly regenerated version was substituted in anyway, to guarantee the FPGA is running the definitively current software build rather than a near-identical one. This is build 24; it produced a clean, comparable timing result to build 23 (WNS=0.000 ns, WHS=+0.003 ns) and was independently reconfirmed at 100% ARVALID reliability.

4.3 Real boot-software console output — checked, not yet observed

Update, rev6: superseded. The console was first observed on 10 October 2026 (build 42) after the SRAM wrapper polarity fix, Section 14.

PMOD2 carries the SoC's real, software-driven UART0 output (distinct from PMOD3 IO[3], the custom debug_uart_probe.sv logger used throughout this whole investigation for hardware correlation captures). Whether real firmware output appears there was checked but not conclusively resolved: a genuine bootloader source exists (megasoc_tech/software/src/bootloader/bootloader.c, which prints a SoCLabs banner) and its content is confirmed embedded in the ROM, but no successful capture of visible output on PMOD2 was recorded in this revision. Given the bootloader's own object files touch the GIC-400, page tables and QSPI flash before reaching any UART print, a hang partway through that sequence is a plausible, untested explanation. This remains open — see Section 8.

5. An independent constraints review, and what it led to

After the ARVALID result was reached, an independent SoC-Labs constraints review of megasoc_haps_sx.xdc was obtained (artifact: “MegaSoC HAPS Constraints Review”). Several of its conclusions were cross-checked directly against this project's own RTL and real hardware/tool behaviour rather than accepted at face value; the review's central claims held up, with one number (an early estimate of ~140 unbuffered clock gates) self-corrected by the review itself once measured directly against a placed checkpoint, and the review acted on that self-correction.

5.1 Headline finding: the same waiver hid a design-wide problem

The review reached, independently and from SDC semantics rather than the empirical route this project had used, exactly the same conclusion as Section 3.1: -hold-class exceptions cover Removal as well as Hold, and the blanket waiver therefore hid every hold/removal check in the design, not only the one path already fixed by the pblock.

5.2 Removing the waiver: what was actually hiding underneath

The blanket waiver was removed (keeping the pblock exactly as-is, as an isolated experiment) and the design rebuilt with real hold/removal analysis for the first time. Result:

Metric	With blanket waiver (all prior builds)	Waiver removed, nothing else changed (build 26)
Failing hold/removal endpoints	0 (hidden, not actually zero)	157,405 – 168,745 (of ~244,000 total)
Total hold violation (THS)	0 (hidden)	−163,734 ns to −186,770 ns
Worst single path (WHS)	0.000 ns (hidden)	−3.719 ns

Vivado's own standard automatic hold-fix pass (phys_opt_design -hold_fix — never once run on this design in its whole history, because hold was always waived) was tried directly against this result and barely moved it (168,745 → 168,706 failing endpoints). This ruled out “ordinary local routing slop” as the explanation and confirmed the problem was structural.

5.3 Root cause 1 of 2: missing ASYNC_REG on the CA53's own synchroniser cell

The single worst path in the whole 168,745-endpoint picture was, itself, informative: not stop_clk_reg, but a different instance of the same underlying RTL cell — ca53_cell_sync.v, a plain 2-flop r0/r1 chain used 36 times across the CPU for exactly this kind of async-to-sync boundary, with no ASYNC_REG attribute anywhere in the RTL or the XDC despite genuinely being a synchroniser (r0's D input changes at the instant its own async clear releases — a textbook metastability-capture flop). A 3,000-path sample of the worst failing endpoints showed 94% landing in one module (u_ca53ifu_pf_bpd, the instruction fetch unit's branch-predictor table) — not because that module has its own defect, but because it happens to be enormous (6,488 flip-flops), so one bad net's fanout lands on thousands of individual destinations that all sit inside it.

Fix (build 27), applied via XDC rather than editing the licensed Arm RTL:

```
set_property ASYNC_REG TRUE [get_cells -hier -filter {NAME =~
"*ca53_cell_sync*/r0_reg" || NAME =~ "*ca53_cell_sync*/r1_reg"}]
```

Metric	Before (build 26)	After ASYNC_REG fix (build 27)	Change
Failing hold/removal endpoints	168,745	41,444	−75%
Total hold violation (THS)	−186,770 ns	−17,026 ns	−91%

5.4 Root cause 2 of 2: clock gates synthesising as LUT-driven global buffers

After the ASYNC_REG fix, the new worst path was of a completely different, ordinary kind (same-clock Hold, not Removal), between two flops with 5.449 ns of pure clock skew between them and no synchroniser involved at all — the signature the constraints review had predicted for its second claim: roughly 20 of this design's 24 global clock buffers (BUFGCE) are fed by combinational LUT logic rather than a clean source. This was fact-checked directly against a routed checkpoint using the review's own Tcl query and confirmed exactly: 20 LUT2-sourced, 3 clean (IBUFCTRL), 1 clean (BSCANE2 — the JTAG TCK, correct by design).

Root of the problem: PREICG_X4B_A9TS_C31.sv, the FPGA implementation of Arm's real ASIC clock-gate cell (instantiated ~100–140 times across the CPU/GIC/SCU), had already been through two prior revisions documented in rev3 — a behavioural latch+AND model (synthesises as a LUT directly in the clock path) and a blanket conversion to real BUFGCE (which exhausted clock-routing tracks and failed placement) — settling on a USE_BUFGCE parameter promoting only 2 of ~140 instances, leaving the rest on the LUT-based fallback. That fallback is exactly what the hold/removal audit was now measuring as a real, previously invisible defect.

A synthesis-level attempt was tried first: Vivado's built-in GATED_CLOCK_CONVERSION=auto option (a different, tool-native mechanism from either prior PREICG revision). It genuinely cleared a related but distinct DRC warning (PDRC-153, 0 occurrences afterward) by restructuring 143 combinational gated clocks onto flip-flop CE pins, but it left the specific LUT2→BUFGCE pattern completely unchanged (still 20 of 24) — the two mechanisms turned out to be independent, and this option was later found to be unstable across unrelated netlist changes (Section 6.4) and was ultimately not relied on.

Fix actually adopted (build 28) — PREICG_X4B_A9TS_C31.sv, revision 0.04, a third option nobody had tried: bypass gating entirely by default. Clock gating has no functional meaning on an FPGA (it is a power-saving feature, irrelevant until an ASIC pass needs it), so there is no correctness reason to keep any instance gated during bring-up, and a straight passthrough needs neither a LUT (no combinational logic left at all) nor a dedicated buffer (no new clock domain is created), sidestepping both of the two previous revisions' failure modes at once:

```
module PREICG_X4B_A9TS_C31 #(
  parameter USE_BUFGCE = 0,
  parameter FPGA_BYPASS_GATING = 1   // NEW, defaults on
```

```
) (
  output wire ECK, input wire E, input wire SE, input wire CK
);
  generate
    if (FPGA_BYPASS_GATING) begin : g_bypass
      assign ECK = CK;
    end else if (USE_BUFGCE) begin : g_bufgce
      BUFGCE u_bufgce (.I(CK), .CE(E | SE), .O(ECK));
    end else begin : g_behavioral
      /* original latch+AND model, preserved, now the fallback */
    end
  endgenerate
endmodule
```

Result (build 28): BUFGCE count across the whole device dropped from 24 to 9. Every single LUT2-driven instance in the CA53/governor/SCU family is gone — replaced either by a plain wire (no buffer needed at all) or, for the handful of genuinely high-fanout reset-distribution nets (the ASYNC_REG-tagged synchroniser outputs from Section 5.3), a clean FDCE-sourced buffer, which is the normal, correct pattern for wide reset fan-out.

Metric	Build 27 (ASYNC_REG only)	Build 28 (+ clock-gate bypass)	Change
Failing hold/removal endpoints	41,444	33,509	−19%
Total hold violation (THS)	−17,026 ns	−13,675 ns	−20%

Cumulative result, build 26 (waiver just removed) to build 28 (both fixes applied): 168,745 → 33,509 failing endpoints (

an 80% reduction

), and −186,770 ns → −13,675 ns total violation (

a 93% reduction

), from two targeted, well-diagnosed, independently-validated fixes — one a one-line XDC attribute, the other a single-file RTL parameter default. Build 28 is the

last known-good, fully rebuildable, stable baseline

 reached in this revision. It still shows 33,509 failing endpoints and is not fully timing-clean, but it builds reliably in normal time (no routing failures, no multi-hour anomalies) and its two fixes are both independently confirmed to be doing exactly what they were intended to do.

6. The QSPI clock-domain-crossing bug — real, partly fixed, not yet closed

With the two dominant causes fixed, the residual 33,509 failing endpoints were checked for concentration again (a 3,000-path sample): no single dominant cluster remained (the largest, QSPI, was 442 of 3,000 — 14.7%), spread thinly across dozens of modules instead. A second phys_opt_design -hold_fix attempt on this improved checkpoint again barely helped (33,509 → 33,363), ruling out ordinary routing slop a second time.

6.1 Root cause

The QSPI flash controller generates its own serial clock, QSPI_SCLK_i, by dividing HCLK with a runtime-writable register (qspi_clock_div.v, ratio set by software at any time):

```
assign QSPI_SCLK_i = (QSPI_CLK_DIV==5'h00) ? HCLK : QSPI_SCLK_reg;   // toggle-flop divider
```

This clock was never declared anywhere in the XDC. With no declaration, Vivado's static timing analysis does not recognise it as a separate clock at all — it traces electrical connectivity through the divider's mux and, for practical purposes, treats registers on QSPI_SCLK_i as if they shared HAPS_CLK's exact edges. This was confirmed directly: a register known from the RTL to be clocked by QSPI_SCLK_i (data_in_reg) showed up in Vivado's own timing report as “clocked by HAPS_CLK.” That mis-analysis is what produced the QSPI cluster of “violations” in the post-waiver-removal picture.

Independently of that constraint gap, the actual RTL crossings between HCLK and QSPI_SCLK_i were found to be genuinely unsafe: only a single flip-flop synchroniser in each direction (QSPI_enable_sync going in; AHB_QSPI_ENABLE_ACK and AHB_QSPI_BUSY consumed raw, with no synchroniser at all, coming back), where a correct crossing needs two stages. This is a real metastability risk independent of any timing-constraint question.

6.2 RTL fix applied — retained, considered correct, not the source of the later problems

Both crossings were widened to proper 2-stage (* ASYNC_REG = "true" *) synchronisers, in qspi_controller.sv (forward) and ahb_qspi_interface.sv (return), with the FSM logic updated to consume the synchronised signals. This RTL change alone (tested in isolation, Section 6.3, build 29) produces only a small, acceptable regression and is considered a genuine, retained improvement regardless of the outcome of Section 6.3–6.4.

6.3 XDC-side fix attempts — three tried, all unsuccessful, in a clear pattern

Getting Vivado to correctly treat the crossing as asynchronous (rather than leave it silently mis-analysed) proved much harder than expected, and produced the clearest and strangest finding of this revision.

Build	QSPI RTL sync (Section 6.2)	XDC treatment of the 3 sync paths	GATED_CLOCK_CONVERSION	Result
28	not present	—	auto	33,509 failing endpoints (stable baseline)
29	yes	create_generated_clock declared at a downstream leaf pin + set_clock_groups -asynchronous — did not propagate (only 1 of 1,434 registers on the net actually resolved to it); effectively no working exception	auto	38,509 (small regression — RTL change alone is nearly harmless)
30 / 30b	yes	set_false_path on each synchroniser's D input	auto	Build 30: routing never completed after 12+ hours, congestion spiking to 100,000+ overlapping nodes, killed. Build 30b: routing completed (~14.5 h) but left 8 nets with unresolved DRC RTSTAT-6 partial-route conflicts; bitgen refused to run.
31	yes	set_max_delay -datapath_only 40 ns on the same 3 paths (a softer exception, chosen specifically to avoid removing all timing pressure, after 30/30b's routing failures)	auto	Builds successfully this time, but 108,204 failing endpoints — a severe regression, over 3× worse than build 28. New LUT2-driven BUFGCEs appear in unrelated GIC-400 logic (FSM_onehot_state_reg[2]_i_2 ×5, priority_state_q_reg[7]_i_3) that were not present in any earlier build.
32	yes	same set_max_delay exceptions as build 31	off (to test whether the auto heuristic was the cause)	111,841 — still severely regressed, slightly worse than build 31. This disproves GATED_CLOCK_CONVERSION as the cause.

Reading across this table, the correlation is specific and repeatable: the RTL synchroniser hardening alone is nearly free (build 29). Adding any XDC timing exception at all on those same three paths — false_path or max_delay, and independent of the GATED_CLOCK_CONVERSION setting — triggers a severe, unrelated regression in logic (GIC-400 priority/state machine gates) that has no direct connection to QSPI. The mechanism for this is not yet understood; the most plausible explanation is that adding any exception at all on these paths perturbs Vivado's timing-driven synthesis/optimisation ordering enough to change unrelated automatic clock-buffer-insertion decisions elsewhere, but this has not been confirmed.

6.4 Current state — genuinely open

The next, not-yet-tried step identified and agreed at the point this revision was written: declare QSPI_SCLK_i as a generated clock correctly, at its true source (the qspi_clock_div toggle register's own output), rather than at a downstream leaf pin as build 29 mistakenly did — which should let it propagate network-wide the way it did successfully for HAPS_CLK itself, and may make a path-specific XDC exception unnecessary altogether. This has not yet been attempted.

Practical consequence for anyone picking this up: the live project source tree, as of this document, is in the state used for build 32 — i.e. it includes the QSPI RTL fix, and the regressed set_max_delay exceptions, with GATED_CLOCK_CONVERSION set to off.

This is not the last known-good state.

Build 28's XDC (waiver removed, ASYNC_REG fix, clock-gate bypass, no QSPI-specific exception at all) is the last configuration confirmed to build cleanly and reliably. Anyone continuing this work should either (a) revert to build 28's XDC as an immediate, safe baseline before attempting the QSPI fix again with the correctly-sourced generated clock, or (b) continue directly from the live tree with that understanding. This decision was still pending when this document was written.

Update, rev5 (30 September): this section is superseded. The QSPI clock was removed altogether rather than constrained: the controller now runs on the system clock with edge enables (Section 9.6). The set_max_delay lines discussed above were later found never to have been in force (Section 9.9).

7. Superseded findings — corrections to this revision's own earlier positions

Recorded explicitly, in the same spirit as rev3 Section 8, because intermediate conclusions reached during this revision's own period were themselves overturned by later evidence within the same period.

Earlier position (this revision)	When	Current position
Reliability with the pblock fix is ~78% (28/36); residual ~22% failures are a real, unexplained hardware defect worth chasing with new debug taps	Early in this revision, reported externally	The pblock fix alone measures 100% (164/164 across two clean captures). The ~22% was a debug-UART capture artefact (multiple orphaned concurrent readers), not a hardware defect. See Section 3.3.
The second ca53_cpu_reset.v hazard and the SCU tag-invalidate handshake are plausible contributors to the residual failure rate	Mid-revision	Both leads are moot — there was no residual failure rate to explain once the measurement bug was fixed. Neither was conclusively confirmed or refuted before this was discovered.
GATED_CLOCK_CONVERSION=auto is likely the cause of the severe regression seen when adding QSPI XDC exceptions	After build 31	Disproved directly: build 32, identical except GATED_CLOCK_CONVERSION=off, regressed just as badly. See Section 6.3/6.4.
A create_generated_clock declared at a downstream consumer pin should correctly describe QSPI_SCLK_i	Build 29	Confirmed not to work — it does not propagate to the rest of the net. Must be declared at the true source instead (Section 6.4, not yet attempted).

8. Project portability — moving the project to a second machine

The project was copied to a Desktop folder for transfer to a different PC (university machine), which surfaced and led to fixing real, pre-existing project-hygiene defects unrelated to anything above, worth recording because they will recur for the next person who copies this project.

8.1 What was copied

haps_top.xpr, haps_top.srcs (all sources, including every locally-materialised file edited in this revision), haps_top.cache, haps_top.ip_user_files and haps_top.gen — 557 MB in total. haps_top.runs (build artefacts/bitstream) was deliberately excluded: it is not needed to rebuild from source, and was mid-write by an in-progress build at the time of transfer.

8.2 Broken fileset entries found and fixed

Opening the transferred project on the second machine produced “could not find the file” errors for five specific files (Axi.v, Axi_undefs.v, AhbDefns.v, p_flash_cache_f0_gen_const_pkg.vh, ca53ifu_defs.v). Root cause: each of these files has a duplicate entry in haps_top.xpr, one portable ($PSRCDIR/...) and one pointing directly at the original Windows machine's path (C:/Users/Tasos/megasoc_tech/...) or the shared /research/AAA/ library — neither of which exists on a different PC. This class of duplication (multiple fileset entries for the same physical file) is pervasive in this project — 136 files in total carry an AutoDisabled="1" duplicate entry — but a systematic check confirmed only these 5 files had the specific, broken pattern of zero enabled copies remaining after the non-portable entries were removed; the other 131 are legitimately, harmlessly disabled (simulation-only testbenches, ASIC-only tsmc180 technology cells, unused alternate peripheral variants).

First fix attempt made this worse: deleting the non-portable duplicate for Axi.v/Axi_undefs.v left only their AutoDisabled="1" portable copy behind, meaning the file existed nowhere active in the synthesis fileset — which surfaced as a second, different error (“cannot open include file”, 40 instances, since 68 NIC-400-generated files include these two headers by bare filename). Corrected by removing AutoDisabled="1" from the surviving portable entries instead of deleting anything further. Verified directly against Vivado's own resolved project state (IS_ENABLED property), not just by re-reading the XML, after both the mistake and the correction.

9. Structured review and consolidation (17–30 September 2026)

Two things happened in this period. First, the project was ported to a second machine (Windows workstation, Vivado 2026.1) and put through a structured review: static reading of every project-authored and hand-forked source, the synthesis and implementation logs, queries against the routed netlist, and a full disassembly of the boot ROM from the reset vector to UART initialisation. That review produced the HAPS Bug Register of 29 September (57 items). Second, an external review note of 23 September proposed a set of fixes that were applied independently on the Linux server. The two lines of work were reconciled on 30 September (Section 9.12).

Unless a subsection says otherwise, every fix in this section is verified by synthesis and implementation reports and by static analysis only. None has been confirmed on hardware.

9.1 Clock gating: bypass extended to the power controllers and the GIC

Section 5.4 bypassed the CA53's own PREICG clock-gate cells. Two further families were still gated: pck600_clock_gate (the PCK-600 power controllers that sequence CPU power-up and reset release) and gic400_inter_clkgate (the GIC-400 distributor). Both kept the behavioural latch-and-AND model, which on FPGA becomes a latch and a LUT in the clock path. Measured on the routed design, 144 of the 200 worst hold paths ended in the power controllers clocked this way; after bypassing both cells, none did. Two defparam lines that tried to force USE_BUFGCE on the PREICG cells were found to be unreachable and were deleted.

9.2 Reset structure

Three resets were formed by combinational ANDs of power-controller outputs and fed straight to asynchronous reset pins: ROM_RESETn, SRAM_RESETn and the CA53's nCORERESET (the latter from the August experiment described in Section 3). A LUT ahead of an async reset can emit a runt pulse when its inputs change on the same edge, and the release edge was unsynchronised. The ROM and SRAM resets now assert asynchronously and release through a two-flop synchroniser on the peripheral clock. The CA53's four reset inputs are driven from a single signal, asserted asynchronously and released together after a 16-cycle hold, as the Cortex-A53 TRM requires. One side effect is recorded: a warm reset now also asserts the power-on and debug resets, so debug state is lost on a warm reset. Acceptable for bring-up; to be reviewed for the ASIC.

9.3 ROM gate request

A debug counter, left over from an earlier investigation, drove the ROM controller's external gate request low for the first 64 cycles after reset release and for 64 of every 512 cycles thereafter, ignoring the accept signal. The first window coincides with the CPU's first fetch and the EL3 page-table walk. The controller's gated response path is configured never to answer, so each window simply stalled ROM traffic. The request is now held at 1 and the counter is gone.

9.4 L2 data RAM model

Update, rev6: the two-cycle model introduced here was itself wrong (one cycle late against Arm's generic model) and was corrected in build 53; it was not the cause of the cache-enable stop, which was the L2 tag RAM integration (Section 18).

The two-cycle SRAM model used for the sixteen L2 data banks advanced its output stage only while the chip enable was active in the same cycle. A read followed by an idle cycle therefore never reached the output; the data stayed in the first stage until the next access. The output stage is now advanced by the previous cycle's enable. Checked in simulation with an isolated read.

9.5 Peripheral subsystem

Three wiring defects fixed: the watchdog's clock, clock-enable and reset inputs were unconnected (interrupt unknown in simulation, dead on FPGA); the AHB default slave sat on a disabled mux port, so every unmapped access in the peripheral window completed OKAY with data 0; and the default-slave select was not qualified by the bus select, which would have hung other targets once the port was enabled. Unmapped peripheral addresses now return a bus error. The expansion-window BVALID tie (a permanent unsolicited write response) was set to 0. The error slave on the DRAM port accepted write data before an address and then hung; it now waits for AWVALID.

9.6 QSPI: the clock removed rather than constrained

Section 6 left the QSPI clock open. Every attempt to describe it in constraints had either failed to propagate, made routing diverge, or (as found in this period) been rejected by the tool without anyone noticing. The clock itself was the problem: a runtime-selected mux between the system clock and a divided flop, clocking the controller on both edges, with 186 latches inferred from incomplete combinational blocks. Measured on the routed netlist, 901 register and latch pins had no clock at all, and after Section 5's fixes all 200 of the worst hold paths ended inside the controller.

The fix removes the clock. The divider now produces a registered SCLK level plus two single-cycle enables on the system clock, asserted on the edge where SCLK changes. Every register that was clocked on an SCLK edge is now clocked on the system clock and enabled by the matching pulse, so it updates at the same instant as before. The passthrough case (divider value 0, SCLK equal to the system clock) cannot be expressed as an enable and now behaves as divider value 1, i.e. half the system clock; this is a behaviour change firmware could notice. All 186 latches were converted to registers or fully specified logic. Result on the routed design: setup went from 0.000 ns with much of the block outside analysis to +25.9 ns on a 50 ns period with every path timed; the design's clock domains went from two to one; inferred latches from 186 to 0. The logic is not simulated: the project testbench cannot reach the flash path. Its first real test is hardware.

9.7 Debug logger

The UART event logger that produced every hardware measurement in Section 3 was itself found to have three defects. It drains one event per line (about 1.38 ms) with strict lowest-ID-first arbitration, and the sticky ARVALID flag sat below two per-transaction signals that are quiet only while the CPU is stuck. The logger therefore went blind exactly when the CPU began fetching, and a starved slot and a never-asserting ARVALID produced identical logs. Separately, the clock-activity flag false-positived after every reset because its synchroniser flops were not reset while its compare register was; and the edge detector's reference register reset to zero while the live bus did not, producing about 10 ms of fabricated events at every reset release. The ARVALID flag is now at ID 0, an overflow flag is reported on ID 1, the detector is armed one cycle after reset, and the synchroniser flops are reset. The 100% figure in Section 3.3 was measured after the capture process was fixed, and the starvation

defect could only have hidden a positive result, so the figure stands; it has not been re-measured on any later build.

9.8 RT clock

The real-time clock fed to the RTC and timers was a counter bit at 305 Hz, where 32.768 kHz is intended, and the one-hertz clock derived from it inside the peripheral subsystem ticked once per 107 seconds. Neither was declared as a clock, so 47 register pins, including the RTC's 32-bit seconds counter, sat outside timing analysis. The top-level divider is now a divide-by-610 toggle register declared as a generated clock. The one-hertz clock inside the peripheral subsystem is still undeclared. Confirmed by ROM disassembly not to affect boot.

9.9 Constraints that were silently not in force

Three cases were found where a constraint had done nothing without anyone knowing. The three set_max_delay -datapath_only lines of Section 6.3 had been rejected on every build for lacking -from (Constraints 18-540), so the experiment that compared them with set_false_path had in fact compared set_false_path with no constraint at all. A Tcl procedure written into the constraints file as a guard against exactly this class of failure was itself rejected by the project flow (Designutils 20-1307), and every constraint that used it was dropped; the guard now lives in the build script, which stops between synthesis and implementation if an expected clock is missing or the log contains a rejected-constraint warning. The CLOCK_DEDICATED_ROUTE property on the clock input buffer finds no object during synthesis and is probably obsolete.

No I/O port had timing constraints. Constraints for the flash pins were added, referenced to a generated clock at the SCLK pad (following D. Mapstone's constraints for the same IP on Pynq-Z2). The first version specified hold requirements the hardware does not need: the controller changes data on the SCLK falling edge and the flash samples on the rising edge, half a period later, but that relationship is guaranteed by the enable structure, not by anything static timing can see, so the tool demanded same-edge hold and reported 319 pins it could not fix. Hold is now excluded on the flash data pins, setup is kept, and chip select is a false path. These constraints are implementation-only, because the pad clock cannot be resolved during synthesis.

9.10 Placement and the remaining hold problem

With the structural sources of hold failure removed, what remains is broad and shallow: about 20,000 endpoints failing by small amounts across the NIC-400, the power controllers, the SD controller and the AXI bridges, which the router reports it cannot fix. The clock utilisation report explains it: the single 20 MHz clock drives about 89,000 loads over 57 to 59 clock regions, and a per-cell placement count shows the design occupying SLR0 and SLR1 with nothing in SLR2 or SLR3, while using 6% of the LUTs and 1% of the registers. The worst individual paths cross the SLR boundary. A router option suggested by the tool itself (global hold iteration) was tested and produced a bit-identical result; post-route physical optimisation with hold fixing recovered about 2 ns.

A floorplan constraint confining the SoC to two SLRs is prepared but not yet enabled. The first draft named SLR1:SLR2 on the assumption that the design spanned all four SLRs; measured directly on this part, clock-region rows Y0–4 are SLR0, Y5–9 SLR1, Y10–14 SLR2 and Y15–19 SLR3, each with 540 block RAM sites, and the design already sits in SLR0 and SLR1. The range was corrected to SLR0:SLR1.

Since the design is already there, a tighter region within those two SLRs is what would reduce the clock span. The 487 block RAMs used are 45% of two SLRs' capacity and 90% of one, which is why one SLR is not an option.

9.11 Methodology audit

A fresh report_methodology on a routed design of this period listed 700 items. Most are pre-existing and benign (216 byte-write-enable notes on the CA53 cache RAMs; 265 large-skew hold paths, all the same pattern as Section 9.10; nine LUT-driven async resets inside licensed Arm RTL). Three are worth recording. The RTC's 32-bit counter had no timing clock at all (32 registers, critical). Vivado's own CDC check flagged the QSPI crossing as unsynchronised, and flagged the pair-wide datapath-only exception that had been tried on the Linux server as a masking risk; both are moot now that the clock is gone. And the first version of the CA53 reset fix on the Linux server still left a two-input LUT driving the core reset pin; the single-signal version adopted in the merge does not.

9.12 Two source trees, and the merge

Between 17 and 29 September the Linux server and the Windows workstation each carried fixes the other lacked, and solved two problems (the CA53 reset, the logger) in different ways. On 30 September the trees were diffed file by file: 38 files differed. The Windows tree was taken as the base, because it carries the QSPI rewrite; the RT clock fix and the build-script guard were carried over from the Linux tree, and the Linux versions of the reset and logger were dropped. The Linux tree's sources were archived before the merge. Which machine builds from here is an open decision; the two should not diverge again.

9.13 Timing progression in this period

Stage	WNS (ns)	WHS (ns)	Latches	Critical warnings
Start of period (Section 5 fixes in place)	0.000	-4.592	9	4
+ power-controller and GIC gating bypassed; resets; ROM stall; L2 RAM; logger; peripherals	0.000	-3.524	7	4
+ QSPI reclocked; all latches removed	+25.909	-0.664	0	0
+ flash I/O constraints; post-route hold fixing	+18.008	-0.856	0	1
+ flash pad hold exclusion (Windows)	running at time of writing		0	
Merged tree + RT clock (Linux, Vivado 2024.1)	+13.309	-0.649	0	0

The 0.000 ns setup figures in the first two rows are not real margins: much of the QSPI block was outside analysis. The +25.9 ns figure in the third row is the first setup result with every path timed. In the merged build the setup-limiting path is the flash input (pad clock to system clock, 13.3 ns); internal setup margin is 21.7 ns. Its worst hold is -0.649 ns, with 44,955 endpoints failing by a total of 13,686 ns, and the router reports 19,665 pins it cannot fix - the same population as on the Windows builds. No LUT remains anywhere on a clock tree.

10. Bug register

One row per bug found in the programme to date, from the first board bring-up to 30 September 2026. Each row gives the bug, its root cause, and the fix where one is implemented. Rows with an empty Fix column are open. The register is organised by area, not by date, and deliberately carries no build numbers or source-tree versions; those are in Sections 3–9 and Appendix B.

10.1 CPU boot

#	Bug	Root cause	Fix
1	CPU never issued its first instruction fetch (ARVALID never asserted); one build worked and a nominally identical one did not.	The execution clock-enable flop in the CA53 governor has an async preset fed by a resynchroniser flop. The Removal check on that path was waived by the blanket hold waiver (row 2), so placement landed on either side of the real margin from build to build.	Placement constraint keeping the two flops together. Confirmed 164/164 boots. Now redundant: the clock-gate bypass (row 6) removed the flop, and the constraint should be deleted.
2	Blanket set_false_path -hold on the only clock.	Added early as "standard for single-clock prototypes". A hold-class exception also covers Removal, so every hold and removal check in the design was waived.	Removed.
3	CA53 two-flop synchronisers (36 instances) could be separated or retimed by the tools.	No ASYNC_REG attribute in the licensed RTL.	ASYNC_REG applied from the constraints file.
4	Boot reliability measured at 78% when it was 100%; weeks spent on two false leads.	Orphaned serial readers on the host split the UART byte stream between them.	Kill stale readers before capture. Clean captures measured 51/51 and 113/113.
5	Real boot console (UART0) never observed on hardware.	Unknown. The boot ROM touches no peripheral before UART init and the banner is due within about 350 us of reset, so silence means the CPU stops early.	

10.2 Clocking

#	Bug	Root cause	Fix
6	CA53 clock-gate cells synthesised as LUTs in the clock path (about 140 instances); the tool promoted some to global buffers fed by a LUT, a different set on each build.	The ASIC clock-gate cell was modelled as a behavioural latch and AND. Clock gating has no function on FPGA.	Gating bypassed (clock passes straight through). Must be reverted for the ASIC.
7	Power-controller (PCK-600) and GIC-400 clock-gate cells: same defect; 144 of the 200 worst hold paths ended in the power controllers.	Same latch-and-AND model, missed when the CA53 cells were converted.	Both bypassed. Must be reverted for the ASIC.
8	Two defparam overrides intended to promote clock gates to global buffers had no effect, and the constraints comments described a clock structure that was not built.	The generate condition they targeted was already taken by another parameter; defparam on a generate condition is not legal.	Deleted.
9	QSPI controller clocked by a runtime-selected mux between the system clock and a divided flop, on both edges; 901 register pins had no clock; every constraint attempt failed or was rejected.	A clock built from fabric logic cannot be described to static timing, and the passthrough arm made the divider flop's clock ambiguous.	Controller reclocked on the system clock with rise and fall enables; passthrough removed (divider value 0 now behaves as 1). Not simulated; first test is hardware.
10	186 latches inferred in the QSPI controller and its register block.	Incomplete always_comb blocks (read data, register readback, last address, FSM next state, acknowledge).	Converted to registers or fully specified logic; zero latches.
11	RT clock 107 times too slow (305 Hz for 32.768 kHz); one-hertz clock ticks every 107 s.	Placeholder divider took a counter bit.	Divide-by-610 toggle register, declared as a generated clock.
12	RT clock and the one-hertz clock unconstrained; the RTC's 32-bit counter had no timing clock at all.	Counter bits used as clocks with no create_generated_clock.	Top-level RT clock declared. One-hertz clock inside the peripheral subsystem still undeclared.
13	Design placed across two SLRs and 57 to 59 clock regions at 6% LUT utilisation; about 20,000 hold endpoints the router cannot fix.	Clock skew scales with clock-tree span; the worst paths cross the SLR boundary. No placement guidance was given.	Floorplan constraint prepared (SLR0:SLR1, corrected from a draft that named the wrong SLRs). Not yet enabled.
14	Simulation and FPGA use different clock topologies.	The behavioural file list drives three unrelated clocks; the FPGA ties CPU, system and flash clocks to one net.	Documented. Check which file list a simulation used before treating it as representative.
15	Constraints file analysed the design at the wrong frequency.	Period written before the board clock was settled.	Period corrected to the programmed clock.

16	ILA and debug hub still present in every build, a second (JTAG) clock domain, despite being recorded as removed.	Instance left in the ROM wrapper under an ifdef.	Removed. The routed design has one clock.

10.3 Reset

#	Bug	Root cause	Fix
17	ROM and SRAM block resets were a LUT AND of power-controller outputs driving asynchronous reset pins; release unsynchronised.	No reset synchroniser; two flops changing on the same edge in opposite directions can produce a runt pulse.	Asynchronous assert, two-flop synchronous release on the peripheral clock.
18	CA53 power-on reset released about 5 cycles before the L2 reset, contrary to the TRM.	Core and L2 power controllers are independent state machines with no ordering between them.	All four CA53 reset inputs driven from one signal: async assert, 16-cycle synchronous hold on release.
19	The interim core-reset workaround made core reset a permanent function of L2 reset, and formed it as a LUT on an async pin.	Combinational AND of two power-controller outputs.	Superseded by row 18.
20	Top-level auto-reset comparator output mixed into an async-reset branch without being in the sensitivity list; board reset used with no debounce and asynchronous release.	Bring-up convenience logic.	

10.4 Memory and Q-channel

#	Bug	Root cause	Fix
21	ROM Q-channel ports tied off; ROM power controller deadlocked; CPU stalled on its first fetch.	Tie-offs in place of the power-controller connection.	Q-channels connected to the ROM power controller.
22	SRAM low-power sequencer could park, blocking all SRAM traffic.	External gate override wired to the live power request instead of held inactive.	Held at 1.
23	ROM stalled for the first 64 cycles after reset (the first-fetch and page-table-walk window) and 12.5% of cycles thereafter.	A debug counter drove the external gate request, ignoring the accept signal; the gated response path is configured never to answer.	Request held at 1; counter removed.
24	Two-cycle L2 data RAM model returned stale data for any read not followed by another access.	Output stage advanced only while the enable was active in the same cycle.	Output stage advanced by the previous cycle's enable.

25	ROM and SRAM controller port widths: write enable 8 bits into a 9-bit port; SRAM data 64 bits into 65-bit ports. The parity lane is undriven while cache protection is enabled.	Wrapper signals not sized for the parity bit.	
26	Memory controller low-power state machine has an unused state that decodes to accept and deny at once, and a default branch assigning X.	Licensed IP; no recovery from an upset.	Not editable. Proposed: bound assertion that the state is never reached.
27	RAM shim outputs tied to 0 where 0 is the asserted level (unverified).	Placeholder tie-offs.	

10.5 Bus and peripherals

#	Bug	Root cause	Fix
28	Expansion-window write response tied permanently valid: an unsolicited response into the interconnect.	Constant tie-off on a stateful signal.	Tied to 0.
29	Any access to the expansion window is accepted and never answered (hangs).	Ready signals tied 1 and valid signals tied 0. The upstream fixes (outstanding-request counters, ID echo) were never ported to the FPGA fork.	
30	Unmapped peripheral addresses completed OKAY with data 0.	AHB default slave on a disabled mux port; default-slave select not qualified by the bus select.	Port enabled; select qualified. Unmapped addresses now return a bus error.
31	Watchdog interrupt unknown in simulation, dead on FPGA.	Watchdog clock, clock-enable and reset inputs left unconnected.	Connected.
32	Error slave on the DRAM port hung after a write whose data arrived before its address.	WREADY asserted unconditionally in idle, so the beat was consumed and the state machine waited for a second last beat.	WREADY waits for AWVALID.
33	DRAM port had nothing answering it.	DDR4 not integrated on the board; the DRAM wrapper was commented out.	Error slave returning DECERR added as a terminator.
34	Interrupt synchroniser into the GIC-400 is a single flop; RTC-domain interrupts cross asynchronously.	One stage on 240 lines. The GIC requires inputs synchronous to its clock.	Not yet ported: the two-stage version exists upstream.
35	Undriven and unconnected ports: SPI readback upper half; USRT divider output; CPU memory-attribute outputs.	Incomplete wrapper wiring.	

36	Console output routed to a debug-only peripheral (USRT0, not UART0); nothing could reach the board pins.	Wrong peripheral in the printf retarget.	Retargeted to UART0.
37	Boot ROM image lagged a firmware fix.	ROM not regenerated after the firmware changed.	Regenerated from the current firmware image.
38	Boot ROM writes to the UART before configuring the pin mux; early output lost on real silicon.	Initialisation order in firmware.	Not confirmed fixed (tracked separately as MGSOC-49).

10.6 Debug instrument and tooling

#	Bug	Root cause	Fix
39	Debug logger could not report ARVALID at the moment it mattered; a starved slot and a never-asserting signal produced identical logs.	One event per line with lowest-ID-first arbitration; the ARVALID flag sat below two per-transaction signals.	ARVALID flag moved to ID 0; overflow flag on ID 1.
40	About 10 ms of fabricated logger events at every reset release.	Edge-detector reference register reset to zero while the live bus did not.	Detector armed one cycle after reset.
41	Clock-activity flag went true on every boot after the first.	Synchroniser flops were not reset while the compare register was.	Flops reset. The flag is now always true under gating bypass and measures nothing; still to be repurposed.
42	Logger loses one edge permanently for a bit that pulses while queued; timestamps taken at FIFO push, not detection.	Per-bit event queue.	Not fixed in the current logger (a whole-bus format that removes the queue was prototyped and set aside).
43	Reset debug tap is a separate AND gate, not a tap on the reset net that reaches the CPU; it cannot fail.	Tap duplicated the logic instead of observing the net.	
44	JTAG/ILA readback corrupt at 20 MHz (frozen probes, glitches on unrelated signals).	Debug hub behaviour at this clock rate; not root-caused.	UART logger used as the only debug channel; ILA removed.
45	Testbench cannot fail: no $fatal or $error; a dead CPU and a clean boot both exit 0. Monitor watches an SRAM address; debug UART unconnected; simulation lacks the FPGA_BUILD define; no flash model.	Bench written for waveform inspection, not regression.	
46	Project would not open on a second machine (missing files, missing include).	Duplicate file entries, one with a machine-specific path and the portable copy disabled.	Duplicates removed; surviving entries enabled.

47	Under Vivado 2026.1: include path pointed at a file; 17 SD-card files set default_nettype none and never reset it, breaking the Arm memory-controller files compiled after them; an old incremental checkpoint was reused; a run property with a space and a value beginning with a dash aborted the script.	Environment differences between tool versions.	All four corrected.
48	Three constraints were never in force: set_max_delay -datapath_only without -from (rejected every build); a Tcl procedure in the constraints file (rejected, and every constraint using it dropped); CLOCK_DEDICATED_ROUTE finding no object.	Critical warnings not checked between synthesis and implementation.	First two removed or moved to the build script, which now stops on a rejected constraint. Third open.
49	Flash pad constraints demanded hold margin the hardware does not need; 319 pins the router could not fix.	Data changes on the SCLK falling edge and is sampled on the rising edge, a relationship guaranteed by the enable structure and invisible to static timing.	Hold excluded on the flash data pins; setup kept; chip select false-pathed.
50	Global hold iteration router option had no effect.	The router gives up on violator count before consulting it.	Ruled out; do not retry.

11. Current status — done and outstanding

Update, rev6: this section is the position on 30 September and is superseded by Section 19.

Task / action	Status	Note
CPU issues its first instruction fetch (MGSOC-33)	DONE	Section 3. 164/164 boots. Not re-measured since early September; the debug logger that measured it has since been corrected (Section 9.7).
Real boot-software console output (UART0)	OPEN	Never observed. The current boot question. Section 4.3 and bug 5.
Design-wide hold/removal audit	IN PROGRESS	Structural causes removed (Sections 5 and 9). Setup met with margin. About 20,000 shallow hold failures remain from clock skew (Section 9.10); floorplan prepared, not tried.
QSPI clock	DONE (unsimulated)	Controller reclocked on the system clock; zero latches; one clock domain (Section 9.6). Hardware test pending.
Reset structure	DONE (static only)	Section 9.2. Hardware test pending.
Debug logger trustworthy	DONE	Section 9.7. Conclusions recorded before 22 September against the old logger should be read with that in mind.
Single source tree	DONE	Merged 30 September (Section 9.12). Decide which machine builds from here.
Fork reconciled with upstream	OPEN	At least two upstream fixes missing (bugs 29, 34).
Testbench able to fail	OPEN	Bug 45.
Tape-out revert register	OPEN	Section 12. No named owner or gate.

12. Updated tape-out revert and review register

Rev3 Section 9's 15 items and rev4's items 16–23 (reproduced in the previous revision) still apply, with the changes below. New items from this period follow.

#	Item	Type	Action before tape-out
16 (changed)	Governor clock-gate proximity pblock	FPGA-only placement constraint	Now constrains one cell and does nothing (the other was optimised away). Remove.
18 (changed)	Clock-gate bypass, now in three cell families: PREICG, pck600_clock_gate, gic400_inter_clkgate	FPGA-only RTL change	All three must be reverted for the ASIC. The originals are kept in the files.
19–20 (changed)	QSPI synchronisers and constraints	RTL / XDC	Superseded: the controller is reclocked on the system clock. The reclocked controller is a functional change (divider value 0 behaves as 1) and must be reviewed for the ASIC, not assumed FPGA-only.
24	Reset synchronisers on ROM and SRAM block resets; single synchronously released CA53 reset	RTL	Retain for the ASIC after review. Note the warm-reset side effect (Section 9.2).
25	ROM external gate request held at 1	RTL	Retain; the counter it replaced was debug logic.

26	AHB default slave enabled; unmapped addresses return a bus error	RTL	Retain. Firmware that touched an unmapped address will now fault; confirm before sign-off.
27	Watchdog clock, enable and reset connected	RTL	Retain.
28	L2 data RAM model output-stage fix	FPGA model	FPGA-only model; no ASIC action. Confirm the foundry macro's read latency matches the CA53's expectation independently.
29	RT clock divide-by-610	FPGA-only	The ASIC has a real 32.768 kHz source; no action.
30	Flash I/O constraints, implementation-only, hold excluded on the data pins	XDC	Board-specific; no ASIC action. Revisit if the divider's enable structure changes.
31	Floorplan constraint (prepared)	FPGA-only	No ASIC action.
32	Boot ROM pin-mux ordering (MGSOC-49)	Firmware	Must be fixed before tape-out; hidden on FPGA.

13. Recommended next steps

Update, rev6: steps 1–3 were carried out in October (Sections 14–18); the floorplan (step 4) was not needed once build 42 met timing; the upstream fixes (step 5) were ported in the 9 October merge. Current recommendations are in Section 20.

1. Program the merged design and capture both UARTs (debug logger on PMOD3 IO[3], console on the UART0 pin) at 115200 8N1 across several auto-reset cycles. This is the first build with a trustworthy logger and the reclocked QSPI, and it answers the boot question directly.
2. If the console is silent, probe the SRAM bus rather than the peripheral window: the boot ROM's first write is to 0x0080_1ff0, followed by a 54-word burst to 0x80500c–0x8050dc. This separates "CPU never fetched" from "CPU ran but the peripheral path is dead".
3. Re-run the ARVALID reliability capture on the same build before treating it as the new baseline.
4. Enable the floorplan constraint (SLR0:SLR1, or a tighter region within them) as a single-variable experiment against the merged build, and check the clock-region count for the main clock drops.
5. Port the missing upstream fixes: the expansion-window counters and ID echo, and the two-stage interrupt synchroniser.
6. Fix the ROM/SRAM parity-lane widths, the one-hertz clock declaration, and the top-level reset structure; then make the testbench able to fail.
7. Decide which machine builds from here and keep one source tree.
8. Fold Section 12 into a tape-out sign-off process with a named owner. The register now has 32 items and no gate.

10.7 Found in October 2026 (builds 38–54)

#	Bug	Root cause	Fix
51	Boot restarted every ~300 µs forever; console never reached. The CPU fetched from ROM, ran enable_mmu, set up the stack and entered __rt_lib_init, whose first stack-based RET popped a zero link register.	FPGA SRAM wrapper connected the memory controller's active-low chip-enable and write-enable straight to the active-high block RAM, so every SRAM write was dropped and every read returned zero. The ROM wrapper inverts; the behavioural SRAM model is active-low, which is why simulation passed. The wrapper also aliased the RAM every 64 KB.	Inversions and address range corrected. Banner on the console on every boot since (10 October).
52	Boot stopped right after the banner; no bus traffic, no exception, no WFI. Present from the first banner (build 42) to build 53. Caches off, everything else worked.	Arm's TSMC example integration of the L2 tag RAMs (written for a 1 MB L2) feeds the RAMs the unmasked set index; this core has a 256 KB L2, and Arm's generic integration masks the index with l2_size. Tags were written and looked up at different RAM words; the first line fill through the L2 after enable_caches() corrupted execution. Only reached with a cache enabled. Found by bisection in RTL simulation of the FPGA design.	Arm's generic L2 tag integration with an FPGA variant of Arm's generic RAM model. Both caches verified on hardware, stock boot sequence end to end.
53	FPGA model of the L2 data RAM macro returned every read one cycle late.	The model had two register stages, from a misreading of the integration comment "two-cycle latency between the enable input and the data outputs"; Arm's generic model is a one-cycle registered read. Masked by row 52 and invisible with caches off.	One-cycle model; matches Arm's generic RAM over 4000 random cycles.
54	SPI_STARTUP has no branch for the on-board Winbond flash and the boot ROM jumps to the flash whether or not start-up succeeded.	Library supports Micron N25Q and SST26VF064B only; main() ignores the return value.	Firmware. A Winbond branch was written, validated step by step on hardware (QE via 50h/31h, 38h, C0h, EBh with mode A0; controller dummy count 6; a 10-clock all-ones transaction before FFh to leave continuous read) and supplied to SoC Labs.

55	Every 1-bit (SPI-mode) write from the QSPI controller reached the flash with the bits of each nibble reversed (0x02 arrived as 0x04).	The register block swaps the nibbles of each write byte for the 4-bit path; the 1-bit path then shifts each nibble out LSB first. In the upstream SoC Labs RTL.	Fixed in the project copy (MSB first within each nibble). Upstream fix pending.
56	Every 1-bit write from the QSPI controller carried one clock too many, so chip select rose off a byte boundary and the flash discarded all page programs and status-register writes. Erase (no data phase) worked, which made the chip look dead.	The data-out state exits on bit-counter value 7; reads use 6. In the upstream SoC Labs RTL. Never seen before because SPI_STARTUP writes only in 4-bit mode; a Winbond part needs a 1-bit status write before it can enter QPI.	Fixed in the project copy; unit simulation exact for 1-, 2- and 16-byte writes; QE set and page program verified on the chip. Upstream fix pending.
57	On-board flash declared locked or faulty: status register 1 read FF on every attempt.	The test command used opcode 09h, a typo for 05h; the line floated high. The chip was healthy throughout (rows 55–56 explain the failed writes).	Corrected; the chip programs and boots. Recorded as a misdiagnosis.
58	QSPI clock output has a zero-width glitch at the start of every transaction.	The reclocked controller muxes two flops that switch on the same clock edge. Introduced by the FPGA reclocking; the original clock-driven controller does not glitch.	Harmless on this board (the flash decodes correctly). FPGA-only; no ASIC action.
59	Any access to the flash window while XIP is inactive is a bus error, and the boot ROM reports every synchronous exception with the same message.	Interface design plus a one-line exception handler; the console could not tell a dead flash from invalid code.	Diagnostic ROMs print ESR/ELR/FAR. Firmware improvement recommended.
60	Monitor sessions were cut short: 12 of 17 blocks programmed, then reset.	The 10 s auto-reset in the FPGA top ran regardless of activity.	A falling edge on the console receive line suspends the auto-reset for about 215 s, renewed per byte. FPGA-only. Bench rule: after a crash, wait or press PB1.
61	Data abort on the first flash-window read after a control-register write.	Device-memory writes to the QSPI controller and the flash cache were not ordered against the following window load.	dsb sy after every control write in the boot software. Firmware.
62	Start-up code never runs scatter-loading: uninitialised globals are not zeroed and initialised data is never copied on silicon. Concrete case: the flash library's enter-QPI opcode, a global initialised to 0x38, lives in SRAM and holds whatever SRAM powers up with.	boot.s goes straight to __rt_lib_init and main; the RW/ZI region is a separate SRAM load region that only the simulation testbench preloads.	Firmware: call __main or copy/zero in boot.s; place RW in the flash load region. FPGA workaround: images zero-padded over ZI, RW preloaded over UART, the opcode written before every flash boot.

63	A system access to the CoreSight debug block (0x0130_0000) returns a bus error; dap_tests aborts.	Open; most likely the debug power domain is never powered up without a debugger.	Open, low priority. Simulation check of the debug power-up proposed.
64	SPI_WAIT_BUSY can hang after a short QSPI command.	It waits for BUSY to rise before waiting for it to fall; a 2-clock QPI command completes before the first status read.	Firmware: wait for not-busy only after a short delay (done in the diagnostic ROMs).
65	uart_tests cannot pass on hardware as written.	Receive buffer is a NULL pointer (handlers write into ROM space); the second half clears the console UART's transmit enable, muting every later printf; UART1 receive is enabled while the banner is still leaving UART0 (framing lost); the console is left at 2 Mbaud.	Test code. A corrected scratch copy passes both directions with interrupts and WFI on the loopback build.
66	The first UART-loaded program faulted at its second instruction; byte pairs 6–7 of each word and 32-bit stores at +4 landed one word up.	The new block-RAM slave advanced its write address on address accept even when no data beat was consumed; a first beat arriving after the address was stored at address+size.	Fixed; testbench extended with late-data cases. FPGA-only.
67	Two tests hung from flash while passing from RAM.	Their flash images were stale builds (July and January) with other UART clock assumptions; the packaging step reused any existing build.	Rebuilt; packaging now rebuilds unconditionally. Process.
68	First flash-window reads of one build did not set the FLASH_AHB access flag in the logger.	Open; minor.	Open.

14. From silence to the banner — the boot loop (1–10 October, builds 38–42)

Rev5 closed with a trustworthy debug logger, a reclocked QSPI controller and a boot console that had never been observed. The first October builds put the merged design on the board. Build 38 ran the whole-bus logger on the merged tree; builds 39–41 added sticky milestone flags (first fetch, first ROM data, first SRAM write, the boot software's first stack push at 0x801BF0, first flash access, first UART register write, first UART byte) and a field that samples the last accepted CPU address every 50 ms.

14.1 What the logger showed

Every boot was identical: the CPU fetched from ROM, wrote to SRAM at the expected address, never fetched the exception vectors, never reached the UART, and the sampled fetch addresses kept returning to the reset path. The boot restarted about every 300 µs with no warm reset and no exception. That pattern — a successful stack push followed by a return to the reset vector — points at a stack read returning zero: the first stack-based RET in __rt_lib_init popped a zero link register and jumped to address 0.

14.2 Root cause: the FPGA SRAM wrapper's polarity

The memory controller drives its chip-enable and write-enable active low. The FPGA block RAM behind it has active-high CS and WREN. The wrapper written in July connected them without inversion, so during every access CS was 0: writes were dropped and the read data was forced to zero. The ROM wrapper inverts the same signals, which is why ROM reads had always worked, and the behavioural SRAM model used in simulation is active low, which is why every simulation passed. The wrapper also indexed the RAM by the wrong address bits, aliasing every 64 KB. Both were corrected and checked in a unit simulation with the controller's control polarity (bug 51).

14.3 Build 42: the first firmware output on this board

Build 42 (9 October, also carrying the Windows Build G merge that closed timing: WNS +21.5 ns, WHS +0.01 ns, one clock domain) printed ***SoCLabs MegaSoC*** on the console at every 10 s auto-reset. It then stopped: no bus traffic for the rest of the window, no exception, no WFI. The next statement in bootloader.c after the banner is enable_caches(). Build 43 (12 October) used a scratch boot ROM with the two cache-enable calls removed and ran straight through to the flash start-up, proving that cache enable was the stop (bug 52, resolved in Section 18) and separating it from everything that followed.

Rule adopted here and kept for the rest of the period: the FPGA boot ROM is generated from a scratch copy of the boot software for each diagnostic build and installed over bootrom.v; the original is kept beside it (bootrom.v.orig_20260812fw) and must be restored for any non-diagnostic build. Every hardware observation records which ROM was in the bitstream.

15. The jump to flash, and a monitor in the boot ROM (12–14 October, builds 43–46)

15.1 What the jump hit

With caches off, build 43 reached ***Flash Enabled…Booting*** and took a synchronous exception at the jump to 0x400000. The flash window returns a bus error for any access while execute-in-place is inactive, and the boot software ignores SPI_STARTUP's return value and jumps regardless (bug 54), while its exception handler prints one line for every cause (bug 59). Build 44's diagnostic ROM printed the controller registers, the flash's JEDEC identity and the exception syndrome: the chip answered EF 60 16 — a Winbond W25Q32FW, a part SPI_STARTUP does not support — the start-up took the Micron path and left the controller in 4-bit mode against a chip still in 1-bit mode, the window read 0xCCCCCCCC, and the fetch faulted as an undefined instruction. That proved the whole path CPU → interconnect → flash cache → QSPI controller → pads → flash die on the first attempt.

15.2 The monitor ROM and the host tool

Builds 45 and 46 replaced the boot ROM with an interactive monitor (UART command set: memory read/write/dump, raw QSPI commands, JEDEC, erase, program with checksum, memory load, checksum verify, jump, and later production-boot variants) and a Python host tool, haps_qspi.py, that drives it over the console port with termios only. Build 46 added a hold-off of the 10 s auto-reset on console activity (bug 60) and a boot software ROM with barriers after every control write (bug 61). UART0 receive was proven here for the first time.

15.3 The controller under unit simulation

To separate controller from chip, the QSPI register block, controller, clock divider and (later) the AHB interface were simulated with xsim against a scripted flash. This found the nibble bit-order defect in 1-bit writes (bug 55) and the SCLK start glitch (bug 58), and confirmed reads exact. Because the clock count was measured in rising edges rather than real pulses, the extra-clock defect (bug 56) was missed until 19 October.

15.4 The flash declared dead — and why that was wrong

On 14 October the chip answered every identification read but no array read, and ignored every write, erase result notwithstanding; status register 1 read FF constantly, unchanged across board power cycles. It was declared locked or faulty and the work moved to plan B (Section 16). On 19 October the status read was found to use opcode 09h, a typo for 05h (bug 57), and the write failures were traced to the controller (bugs 55–56). The chip was healthy throughout. The misdiagnosis cost no board time — plan B was needed anyway for RAM-resident tests — but it is recorded in full.

16. Running the SoC Labs tests from RAM over UART (14–15 October, builds 47–48)

16.1 A block-RAM slave on the DRAM port

The interconnect's DRAM target (0x8000_0000, mapped Normal, cacheable, executable by the boot page tables) had an error slave behind it. Build 47 replaced it with a 512 KB AXI4 block-RAM slave written for the purpose (one array per byte lane, because Vivado caps a single inferred array at 1 Mbit; INCR, WRAP and FIXED bursts; one outstanding read and write; unit testbench). Build 48 fixed a first-beat address bug found on the board (bug 66), the testbench now covering data beats arriving after the address.

16.2 Images and loading

A new scatter file places a test's code and data in one region at 0x8000_0000 with stack and heap in SRAM; fromelf produces a flat image, zero-padded over the uninitialised-data region because the start-up code never zeroes it (bug 62). The host tool loads the image through the monitor in 4 KB blocks with checksums and jumps to it. The runner mode (suite) does this for a list of images unattended, waits silently through the board's hold-off between tests, follows a console speed change if a test makes one, and writes a summary and per-test logs.

16.3 Results

15 October, build 48: hello_world ran to ** TEST PASSED ** — the first SoC Labs test completed on this board. The full set the same day:

Test	Result from RAM	Note
hello_world, mem_tests, timer_tests, neon_tests, peripheral_test	PASS	
gic_tests	PASS	Timer interrupts through the GIC-400 to the core and its handler: first hardware evidence for the interrupt path and the two-stage synchroniser.
low_power_tests	PASS	Core retention with wake by timer interrupt; ROM power domain switched off and on through its PPU: validates the Q-channel and PPU fixes on the board.
dap_tests	FAIL (abort)	Debug block read returns a bus error (bug 63).
sdio_tests	FAIL (abort)	No card fitted.
pl011_uart_tests	FAIL	Needs a transmit-to-receive loopback that only the testbench provides (Section 17.5).
uart_tests	HANG	Needs UART0/UART1 cross-wiring (Section 17.5) and has four bugs of its own (bug 65).

16.4 A silicon-relevant finding: scatter-loading never runs

Reading the disassembly to understand why consecutive tests could inherit each other's RAM contents showed that the SoC Labs start-up code never calls the scatter-loader: uninitialised data is

never zeroed and initialised data in SRAM is never set on silicon. The simulation testbench hides both. The concrete case in the boot path is the flash library's enter-QPI opcode, a global initialised to 0x38 but living in uninitialised SRAM, so a flash boot with an SST part would send a random command (bug 62, N35).

17. The flash path: emulator, controller bugs, physical flash (15–20 October, builds 49–52)

17.1 An in-FPGA flash emulator (build 49)

With the chip believed dead, build 49 placed an SST26VF064B-compatible flash model on the controller's pad-side signals inside the technology wrapper, backed by a second 512 KB window of the RAM slave reachable over UART. Everything on the SoC side of the pins — controller, flash cache in bypass, XIP window, the unmodified SPI_STARTUP taking its tested SST branch — was then exercised without a working chip. The emulator was first simulated against the real controller stack (zero errors), then a RAM-resident probe on the board read the whole image back through the window with 8-, 32-, 64- and 128-bit loads, including line-crossing ones, with zero errors. All seven passing tests passed from the emulated flash. Switchable with one parameter (USE_FLASH_EMU, now 0).

17.2 The controller's 1-bit write path (build 50)

A review of the flash evidence on 19 October found the status-read typo (bug 57) and prompted a count of real clock pulses per transaction in the controller simulation: 17 instead of 16 for the QE status write, 161 instead of 160 for a 16-byte page program (bug 56), with the data decoded as 0x04 for 0x02 (bug 55). Both defects were reproduced on SoC Labs' original, clock-driven controller RTL, so they are upstream and shared with the ASIC; they never showed because SPI_STARTUP writes only in 4-bit mode. Build 50 carried the fixes and switched back to the real pins. On the board the QE write succeeded at the first attempt and a 16-byte page program read back exactly: the first data ever programmed into the chip from the SoC.

17.3 The Winbond start-up, validated step by step

Register-level sessions through the monitor then validated the start-up sequence drafted for SoC Labs: 38h accepted (a 1-bit JEDEC read afterwards returns FF, as a chip in QPI mode should), JEDEC in 4-bit mode returns EF 60 16 (all four data lines work), C0h, 0Bh and EBh reads exact, the XIP window in continuous-read mode returns the data. Two corrections to the draft came out of it: the controller's dummy count for EBh is 6, not 8, because the chip counts the two mode-byte clocks within its eight; and leaving continuous-read mode needs a 10-clock all-ones transaction before FFh, since 2-clock commands are taken as address nibbles. A third gotcha: the software reset in the exit sequence clears the volatile QE bit, so QE must be set again before every 38h. hello_world booted from the physical flash by hand on 19 October, with the flash cache enabled by the jump.

17.4 Builds 51 and 52: automation and loopbacks

Build 51's monitor ROM runs the corrected Winbond start-up at every reset and boots whatever image is in the flash; the runner's --winbond mode programs each test into the chip over UART

(erase 1.3 s, program 7 s for 68 KB, verified through the 1-bit window), preloads the SRAM data and boots. 20 October: 7 of 7 from the physical flash, dummy count 6 on the first attempt every time.

17.5 UART loopbacks and the uart_tests bugs (build 52)

The two UART tests rely on wiring the simulation testbench provides: UART0 transmit to UART1 receive and back, and PL011 transmit to receive. Build 52 adds the same under the FPGA-only define; UART0's receive is the AND of the host line and UART1's transmit, both idle high and never active together, so the console and the loader keep working. pl011_uart_tests then passed from the real flash. uart_tests still failed, and probes (polled transfers both ways, and the same string sent five ways) showed the cross-wiring and both receive interrupts working while the test itself has four defects (bug 65): a NULL receive buffer, the console transmitter disabled in its second half, the second receiver enabled while the banner is still shifting out, and a 2 Mbaud console. A scratch copy with those corrected passes both directions with interrupts and WFI.

18. The cache-enable stop (20–21 October, builds 53–54)

18.1 First attempt: the L2 data RAM model

The FPGA models of the Cortex-A53's TSMC cache RAM macros are the only part of the core that is not Arm's RTL, so they were the first suspect. The L2 data RAM model had a two-register read pipeline where Arm's generic model has one (bug 53); a unit comparison over 4000 random cycles showed 1370 mismatches against zero for the corrected model. Build 53 carried the fix. On the board the stop was unchanged.

18.2 Simulating the FPGA design itself

Rather than guess further, the exact Vivado file list (769 files, exported from the project) was compiled in VCS with a 20 MHz differential clock, the reset button, pulled-up flash pins and a testbench that prints every byte the software writes to UART0's data register. A boot ROM built from a stepwise probe — instruction cache on, off, data cache on with memory tests, both on — ran in 45 s of CPU time per 400 ms of SoC time and reproduced the board exactly: the console stopped the moment the instruction cache was enabled. With the simulated RAMs preset to zero, as FPGA block RAM powers up, the match with hardware was complete.

18.3 Bisection and root cause

Swapping the cache RAM models for Arm's generic ones made the simulation pass. All eleven FPGA macro models matched Arm's generic RAM cycle for cycle in isolation, and presetting the tag RAMs to all ones (Arm's generic model does so; an all-zero tag reads as valid) changed nothing, so the difference was in the integration files, not the models. Swapping them one at a time showed that only the L2 tag RAM integration decides pass or hang. Arm's TSMC example for this macro library is written for a 1 MB L2 and feeds the tag RAMs the unmasked set index; this core is configured with a 256 KB L2, and Arm's generic integration masks the index with l2_size. Tags were written and looked up at different RAM words, so the first instruction-cache line fill through the L2 after enable_caches() corrupted execution (bug 52). The same file had been in every build since 42. It is FPGA-only: silicon uses SoC Labs' own 256 KB integration.

Build 54 uses Arm's generic L2 tag integration with an FPGA variant of Arm's generic RAM model (defined power-up contents, output held through non-read cycles, synthesisable). The full stepwise program passed in simulation first — instruction cache, data cache, both, 64 KB memory tests at zero errors — then the build was launched. One false alarm along the way is recorded: a data-cache test appeared to freeze in simulation until the simulated RAM slave was zero-initialised; never-written words had read back as undefined values.

18.4 Confirmation on hardware, 21 October

Program (RAM-resident, build 54)	Result
cache_steps: IC IALLU; SCTLR.I on + code loop; I off; C on + 1 KB SRAM + 64 KB RAM; C off; C and I on + 64 KB RAM	PASS, 0 errors in every memory test
cache_probe: the stock enable_caches() of bootloader.c, 8 KB SRAM, two 64 KB RAM tests, code loop, disable, retest	PASS
cache_flashboot: enable_caches at EL3 and EL1 as bootloader.c, Winbond start-up, jump to the image in the physical flash	PASS — Hello SoCLabs MegaSoC, TEST PASSED with both caches on
Regression: seven tests from the physical flash (suite --winbond)	7 of 7 PASS

The stock boot sequence — reset, boot ROM, flash start-up, cache enable, jump into the XIP window — now runs end to end on the board.

19. Current status (21 October 2026) — supersedes Section 11

Area	Status	Evidence
CPU first fetch, ROM, SRAM, boot console	DONE, on hardware	Banner every boot since build 42 (Section 14).
UART0 both directions; UART1 and PL011 loopbacks	DONE, on hardware	Monitor and loader; pl011_uart_tests PASS; corrected uart_tests PASS (Section 17.5).
Interrupts through the GIC-400; PPU power sequencing	DONE, on hardware	gic_tests and low_power_tests PASS from RAM and flash (Section 16.3).
QSPI controller, flash cache, XIP window	DONE, on hardware	Emulated and physical flash; probe at all access widths (Section 17).
Physical on-board flash: program, boot, execute	DONE, on hardware	7 of 7 tests from the chip (builds 51, 54).
CPU caches; stock boot sequence	DONE, on hardware	Build 54, Section 18.4.
Timing	DONE	All constraints met since build 42; build 54 WNS +14.07 ns, WHS +0.01 ns, 0 critical warnings, 0 latches, one clock domain.
dap_tests (debug block read)	OPEN, low	Bug 63 / N36.
sdio_tests	N/A	No card fitted.
Upstream items for SoC Labs	OPEN	Bugs 54, 55, 56, 62, 64, 65 and the TSMC L2 example integration note (Section 20).
Jira	OPEN	Atlassian unreachable from the server since 14 October; all updates paste-ready in the Desktop log.

19.1 Test matrix, build 54

Test	From RAM (UART)	From emulated flash	From physical flash	Note
hello_world	PASS	PASS	PASS (also with both caches on)	
mem_tests	PASS	PASS	PASS	
timer_tests	PASS	PASS	PASS	
neon_tests	PASS	PASS	PASS	
peripheral_test	PASS	PASS	PASS	
gic_tests	PASS	PASS	PASS	
low_power_tests	PASS	PASS	PASS	
pl011_uart_tests	—	—	PASS	Build 52 loopback.
uart_tests	PASS (corrected copy)	—	HANG (as written)	Test bugs, bug 65.
dap_tests	FAIL	—	—	Bug 63.
sdio_tests	FAIL	—	—	No card.
coremark, dma350_tests	—	—	—	Do not build upstream.

19.2 Timing progression, October builds

Build	WNS (ns)	WHS (ns)	Note
42	+21.5	+0.01	Build G merge; first banner
48	+19.447	+0.009	512 KB RAM slave
49	+29.32	+0.01	Flash emulator
50 / 51	+15.067	+0.010	Controller write fixes; ROM v5.2
52	+20.349	+0.010	UART loopbacks
53	+16.332	+0.010	L2 data RAM model
54	+14.072	+0.010	L2 tag integration; 743 → 703 block RAM tiles

20. Tape-out revert register additions, and recommended next steps

Items 1–32 (Sections 12 and the earlier revisions) still apply with the changes noted. New items from this period:

#	Item	Type	Action before tape-out
33	SRAM wrapper polarity and address range (SRAM/fpga/SRAM_wrapper.sv)	FPGA-only	No ASIC action; the behavioural and ASIC SRAM paths are correct.
34	bootrom.v holds a monitor/diagnostic ROM (v5.2)	FPGA image	Restore bootrom.v.orig_20260812fw, regenerated from the current bootloader, for any deliverable build.
35	512 KB block-RAM slave on the DRAM port; flash emulator (USE_FLASH_EMU = 0) and its backdoor	FPGA-only RTL	No ASIC action; remove or keep behind the define.
36	QSPI controller 1-bit write fixes (data phase ends on the byte boundary; MSB first within nibbles)	RTL, shared with the ASIC	Must go upstream and into the ASIC netlist; otherwise no 1-bit status write or page program works from the SoC.
37	UART0/UART1 and PL011 loopbacks under FPGA_BUILD	FPGA-only	No ASIC action.
38	Cache RAM FPGA models (defined power-up, hold through writes, 1-cycle L2 data); Arm generic L2 tag integration with ca53_generic_ram_fpga.v	FPGA-only	No ASIC action; the ASIC uses SoC Labs' mem_integrations. Note for any other FPGA port of this core: Arm's TSMC example L2 tag integration is for a 1 MB L2 only.
39	Console-activity hold-off of the auto-reset; whole-bus debug logger taps	FPGA-only	No ASIC action.
40	Winbond branch for SPI_STARTUP (dummy count 6; 10-clock exit); return value checked before the jump	Firmware	Adopt upstream if a Winbond part is a target; at minimum check the return value.
41	Scatter-loading never runs; SPI_WAIT_BUSY race; uart_tests defects	Firmware / test code	Fix upstream before silicon bring-up (bugs 62, 64, 65).

20.1 Recommended next steps

1. Send SoC Labs the two controller fixes with the simulation evidence (pulse counts and decoded bytes on the original RTL), the Winbond branch with its two corrections, and the firmware and test-code items; the package on the Desktop (for_daniel_flash) holds the first two groups.

2. Paste the October Jira updates (MBT-23…36 status changes; N24–N40) when Atlassian is reachable; close MBT-26 with the real root cause (L2 tag integration, not the data RAM model).

3. Decide the deliverable ROM: restore the original boot ROM, or adopt the Winbond-capable start-up, and rebuild once for the record.

4. Low priority: simulate the debug power-up to settle dap_tests (bug 63); try a card for sdio_tests.

5. Keep the RTL simulation of the FPGA design (scratchpad n25/sim: files.f from the Vivado list, tb_haps.sv) as the first tool for any future FPGA anomaly; it reproduces the board at 45 s per 400 ms and found the cache stop in one night after a week of hardware-only probing.

Appendix A — Environment reference

Item	Value
Development server	srv03335.soton.ac.uk (account ad1u25), Vivado 2024.1
Workstation	Windows, Vivado 2026.1; project under C:\Users\Tasos\Desktop\haps_top_project
FPGA working area (server)	/home/ad1u25/megasoc_project/FPGA_Tasos/haps_top/
Repository	git.soton.ac.uk/soclabs/megasoc_project
Target	xcvu19p-fsva3824-2-e; 4 SLRs of 45 clock regions each (rows Y0–4, 5–9, 10–14, 15–19); 540 RAMBFIFO36 sites per SLR
Design clock	HAPS_CLK, 20 MHz (50.000 ns period)
Debug UART logger	debug_uart_probe.sv, PMOD3 IO[3] (BF19), 115200 8N1; the sole debug channel
Jira	soclabs.atlassian.net — projects MGSOC and MBT (bug tracker)
Bench PC	haps-dev: console /dev/ttyACM0 (UART0, 115200 8N1), logger /dev/ttyUSB0; ~/script/haps_qspi.py (cmd, cmds, program, load, go, boot, suite [--flash|--winbond] [--baud-after], term), ~/script/haps_capture.sh, ~/script/haps_parse_logger.py --map 44
Board	Programmed from srv03335 (ConfPro test_reset_timing.csprj, GCLK1 20 MHz; Vivado hardware manager). On-board flash W25Q32FW on bank 21 (CA49 CLK, CB49 CS, CC49/CC50/CA51/CB51 D0–D3)
RTL simulation of the FPGA design	VCS 2022.06; scratchpad n25/sim: files.f (Vivado compile order + unisims IBUFDS/BUFG/BUFGCE + glbl), tb_haps.sv (UART0 data-register tap, bus monitors); ~45 s CPU per 400 ms of SoC time

Appendix B — Build index

Builds 19–32 (29 July – 17 September) are listed in rev4 Appendix B and not repeated. Builds since (all archived with bitstream, reports and notes under FPGA_Tasos/haps_top/bitstreams/ from build 43 on):

Build	Machine	What changed	Outcome
33	Linux	QSPI generated clock at its true source + set_clock_groups -asynchronous	Routing failure (partial-route conflicts) after ~11 h
34	Linux	Same clock + set_max_delay -datapath_only 200 ns (no -from)	Built; 115,442 failing endpoints
Pre-A	Windows	Port to Vivado 2026.1 only	WHS -4.592; 9 latches
A	Windows	Gating bypass on PCK-600/GIC-400; resets; ROM stall; L2 RAM; logger; peripherals	WHS -3.524
B	Windows	QSPI reclocked; all latches removed	WNS +25.909, WHS -0.664, 0 latches, 0 critical warnings. Recommended for first hardware test
C, D	Windows	+ flash I/O constraints; post-route hold fixing; global hold iteration	WHS -0.856; hold iteration had no effect
E	Windows	Flash pad hold exclusion	Running at time of writing

36	Linux	Independent application of the 23 September proposal + QSPI clock at register output with pair-wide exceptions	WNS 0.000, WHS -3.940, 26,980 failing endpoints
36B	Linux	As 36 with the QSPI clock declared at the mux output and a pad clock	WHS -7.604 (pad hold artefact), 22,297 failing endpoints
37	Linux	Merged tree: Windows E sources + RT clock fix	WNS +13.309 (flash input path; internal +21.7), WHS -0.649, 44,955 failing hold endpoints, 0 latches, 7 clock buffers all flop- or pad-driven. Bitstream archived as haps_top_build37_merged_20260930.bit
38–41	Linux	Whole-bus logger with sticky milestones and sampled fetch address	Located the boot loop: SRAM reads return zero (bug 51)
42	Linux	Windows Build G merge (timing closed) + SRAM wrapper polarity fix	WNS +21.5, WHS +0.01. First banner on the console, 10 Oct. Stops at enable_caches()
43	Linux	Caches-off diagnostic ROM; logger bus map 43	Reaches the flash jump; synchronous exception at 0x400000
44	Linux	Diagnostic ROM v2: JEDEC, registers, exception syndrome; stuck indicators	Flash answers EF 60 16 (Winbond); controller QPI vs chip SPI; UDF at 0x400000
45	Linux	Monitor ROM + haps_qspi.py host tool	UART RX proven; sessions cut by the 10 s auto-reset
46	Linux	Auto-reset hold-off on console activity; ROM v3 (barriers)	Interactive sessions; flash writes still ignored
47	Linux	512 KB block-RAM slave on the DRAM port; dram scatter file	Load works; execution faults at +4 (first-beat bug)
48	Linux	RAM slave first-beat fix	WNS +19.447. hello_world PASS from RAM, 15 Oct; 7 of 11 tests PASS
49	Linux	In-FPGA SST26VF064B flash emulator; ROM v4 production boot	WNS +29.32. 7 of 7 from the emulated flash via unmodified SPI_STARTUP
50	Linux	Controller 1-bit write fixes; real flash pins; ROM v5 with Winbond start-up	WNS +15.067. QE set, page program OK; hello_world from the physical flash by hand, 19 Oct
51	Linux	ROM v5.2: corrected Winbond start-up (dummy 6, robust exit), auto-boot	7 of 7 from the physical flash, 20 Oct
52	Linux	UART0/UART1 cross-wiring, PL011 loopback	WNS +20.349. pl011_uart_tests PASS; uart_tests defects identified
53	Linux	L2 data RAM model 1-cycle	WNS +16.332. Cache stop unchanged
54	Linux	Arm generic L2 tag integration + ca53_generic_ram_fpga.v	WNS +14.072. Both caches work; stock boot from the physical flash, 21 Oct; 7 of 7 regression